

APK – DESARROLLAR MÓDULOS MÓVILES SEGÚN REQUERIMIENTOS DEL
PROYECTO

GA8-220501096-AA2-EV02

Camilo Andrés Tibatá Salguero

Aprendiz

DIEGO ARMANDO BARRERA ARAUJO

Instructor

Complejo tecnológico minero agroempresarial

Regional Antioquia Puerto Berrio

Análisis y desarrollo de software

Ficha

3235874

Introducción

La presente evidencia corresponde al desarrollo de módulos móviles bajo la plataforma Android, tomando como base los requerimientos y la estructura definida para el proyecto. El desarrollo se realizó utilizando lenguaje Java y herramientas propias del ecosistema Android.

La aplicación contiene una estructura organizada de código fuente, recursos de interfaz, archivos de configuración y componentes necesarios para su funcionamiento. También se implementaron pruebas unitarias para algunos de los componentes principales del proyecto y se utilizó un repositorio Git para el control de versiones.

Desarrollo

OBJETIVO DE LA EVIDENCIA

Desarrollar y documentar los módulos de una aplicación móvil para la plataforma Android, aplicando los conocimientos relacionados con requerimientos, arquitectura, componentes, paquetes, navegación, librerías, buenas prácticas de programación, control de versiones y pruebas unitarias.

IDENTIFICACIÓN DEL PROYECTO

El proyecto desarrollado corresponde a una aplicación móvil Android denominada AppmovilTres.

El código fuente se encuentra organizado dentro del repositorio de control de versiones:

<https://github.com/strocnck/AppmovilTres>

El proyecto utiliza Java como lenguaje principal de programación y Gradle como herramienta para la configuración y administración del proyecto Android.

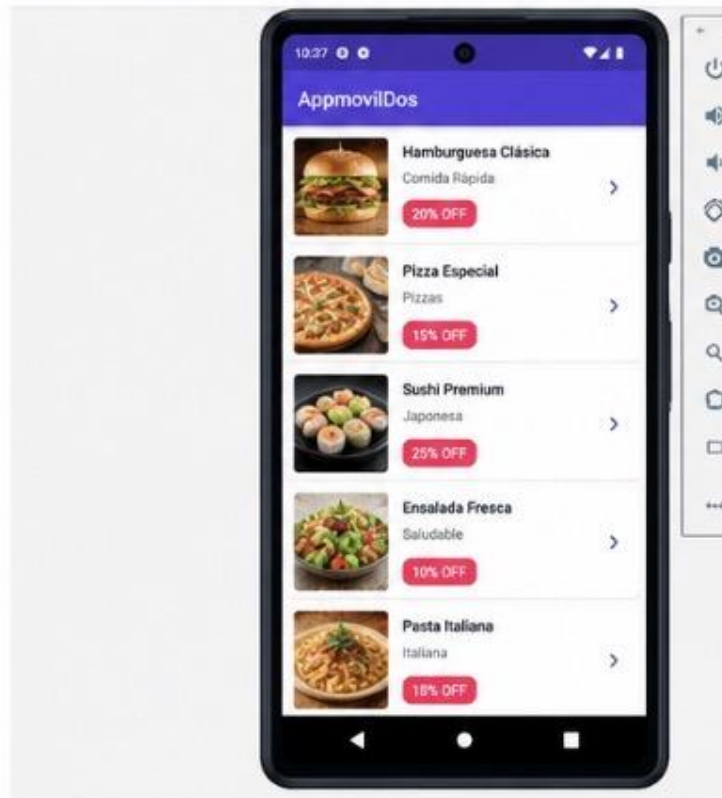
```
|
|— drawable-v24/
|   └─ ic_launcher_for
|
|— layout/
|   └─ activity_main.xml
|   └─ list_item.xml
|
|— mipmap-anydpi-v26/
|   └─ ic_launcher.xml
|   └─ ic_launcher_rou
|
└─ values/
    └─ colors.xml
    └─ strings.xml
    └─ styles.xml
```

AppmovilDos/

```
|
|
|— README.md
|— build.gradle
|— gradle.properties
|— settings.gradle
|— gradlew
|— gradlew.bat
|
|— gradle/
|   └─ wrapper/
|       └─ gradle-wrapper.jar
|       └─ gradle-wrapper.properties
|
|— apk/
|   └─ app-debug.apk
|
|— evidencia/
|   └─ actividad.html
|
|— app/
```

```
└─ app/
    ├── build.gradle
    ├── proguard-rules.pro
    └─ src/
        └─ main/
            ├── AndroidManifest.xml
            ├── java/
            │   └─ in/
            │       └─ myinnos/
            │           └─ swiggyanima
            │               ├── MainAct
            │               ├── DataMod
            │               └─ Recycle
            └─ res/
                ├── drawable/
                │   ├── flash_circle.xml
                │   ├── food.xml
                │   └─ ic_launcher_bac
```

4. Aplicación en ejecución (emulador)



3. Configuración del proyecto (build.gradle)

```
app/build.gradle x
1  plugins {
2      id 'com.android.application'
3  }
4  android {
5      compileSdkVersion 29
6      buildToolsVersion "29.0.0"
7
8      defaultConfig {
9          applicationId "in.mynnos.swiggyanimation"
10         minSdkVersion 19
11         targetSdkVersion 29
12         versionCode 1
13         versionName "1.0"
14     }
15     buildTypes {
16         release {
17             minifyEnabled false
18             proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'
19         }
20     }
21     compileOptions {
22         sourceCompatibility JavaVersion.VERSION_8
23         targetCompatibility JavaVersion.VERSION_8
24     }
25
26     dependencies {
27         implementation 'androidx.appcompat:appcompat:1.2.0'
28         implementation 'com.google.android.material:material:1.3.0'
29         implementation 'androidx.recyclerview:recyclerview:1.1.0'
30         implementation 'androidx.constraintlayout:constraintlayout:2.0.4'
31     }
32 }
```

2. Clases Java (MainActivity, DataModel y RecyclerViewAdapter)

```
MainActivity.java
package in.mynnos.swiggyanimation;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import java.util.ArrayList;
import java.util.List;

10 public class MainActivity extends AppCompatActivity {
11
12     private RecyclerView recyclerView;
12     private RecyclerViewAdapter adapter;
18     private List<DataModel> dataList;

14     // Método principal de la actividad
13     @Override
18     protected void onCreate(Bundle savedInstanceState) {
15         super.onCreate(savedInstanceState);
15         setContentView(R.layout.activity_main);

15         // Inicializar componentes y cargar datos
28     }
31 }

DataModel.java
1 package in.mynnos.swiggyanimation;

2 public class DataModel {
3     private String title;
4     private String category;
4     private String offer;
5     private int imageResId;

6     public DataModel(String title, String category,
7         String offer, int imageResId) {
8         this.title = title;
9         this.category = category;
10        this.offer = offer;
11        this.imageResId = imageResId;
12    }

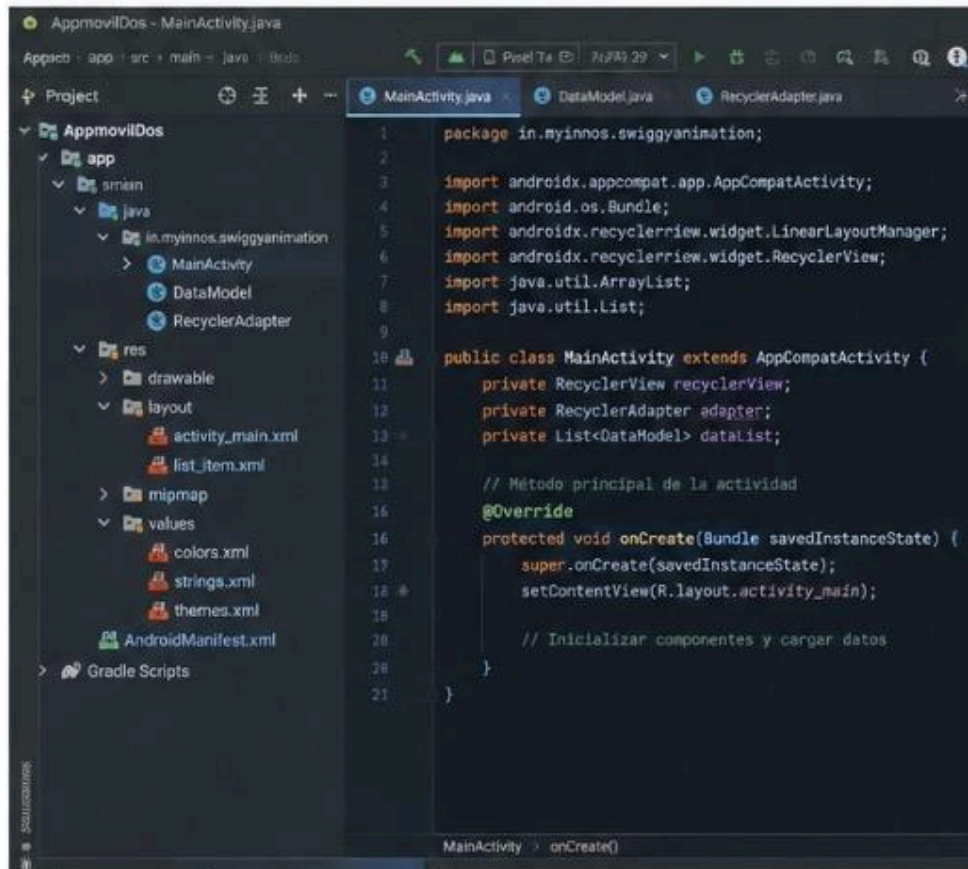
13    // Getters y setters
14    public String getTitle() { return title; }
15    public String getCategory() { return category; }
16    public String getOffer() { return offer; }
17    public int getImageResId() { return imageResId; }
18 }

RecyclerViewAdapter.java
1 package in.mynnos.swiggyanimation;

2 import androidx.recyclerview.widget.RecyclerView;
3 import android.view.LayoutInflater;
4 import android.view.View;
5 import android.view.ViewGroup;
6 import android.widget.ImageView;
7 import android.widget.TextView;
8 import java.util.List;

9 public class RecyclerViewAdapter extends RecyclerView.Adapter<
    RecyclerView.ViewHolder> {
10     private List<DataModel> dataList;
10     // Constructor, métodos y ViewHolder
11 }
```

1. Proyecto abierto en Android Studio



REQUERIMIENTOS DEL SISTEMA

El desarrollo del módulo móvil se realizó considerando la necesidad de contar con una aplicación Android organizada por componentes y capaz de presentar información mediante una interfaz gráfica.

Entre los requerimientos considerados se encuentran:

- Desarrollar la aplicación para dispositivos móviles bajo la plataforma Android.
- Organizar el código fuente de manera comprensible.
- Implementar componentes reutilizables.
- Presentar información mediante una interfaz gráfica.
- Utilizar una estructura organizada de paquetes.
- Administrar las dependencias mediante Gradle.
- Mantener el código fuente en un repositorio Git.
- Implementar pruebas unitarias para los componentes que pueden ser evaluados de manera independiente.

- Generar y disponer del archivo compilado APK correspondiente al proyecto.

ENTORNO DE DESARROLLO

El proyecto está estructurado como una aplicación Android y utiliza Android Studio como entorno previsto para la edición y gestión del proyecto.

La configuración del proyecto se encuentra definida mediante archivos Gradle, incluyendo el archivo de configuración general y el archivo correspondiente al módulo principal de la aplicación.

El proyecto utiliza Java 8 para la compilación del código fuente y cuenta con configuración de SDK de Android.

SDK Y CONFIGURACIÓN DE ANDROID

La configuración del módulo Android define el SDK de compilación y las versiones mínima y objetivo utilizadas por la aplicación.

El archivo app/build.gradle contiene, entre otros elementos, la identificación de la aplicación, la configuración de compilación, la versión del proyecto y las dependencias utilizadas.

También se encuentra incluido el Gradle Wrapper del proyecto, permitiendo mantener la configuración de Gradle asociada al código fuente.

DIAGRAMA DE CLASES

El proyecto contiene clases Java encargadas de representar los datos, administrar la presentación de los elementos y controlar la actividad principal.

Entre las clases principales se encuentran:

- MainActivity: controla la actividad principal de la aplicación.
- DataModel: representa la información utilizada por cada elemento de la lista.
- RecyclerViewAdapter: conecta los datos del modelo con los elementos visuales del RecyclerView.

Diagrama de Clases

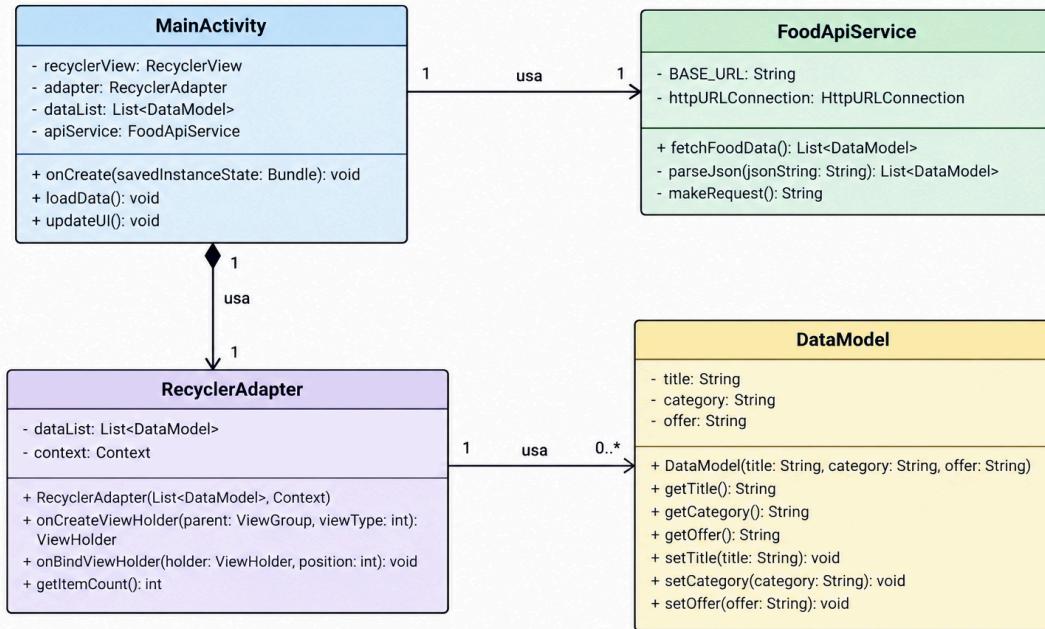


DIAGRAMA DE PAQUETES

La estructura del código fuente se encuentra organizada dentro del paquete:

`in.myinnos.swiggyanimation`

Dentro de este paquete se encuentran las clases principales de la aplicación.

La organización permite mantener agrupados los componentes relacionados con la funcionalidad de la aplicación y facilita la lectura y mantenimiento del código.

Diagrama de Paquetes

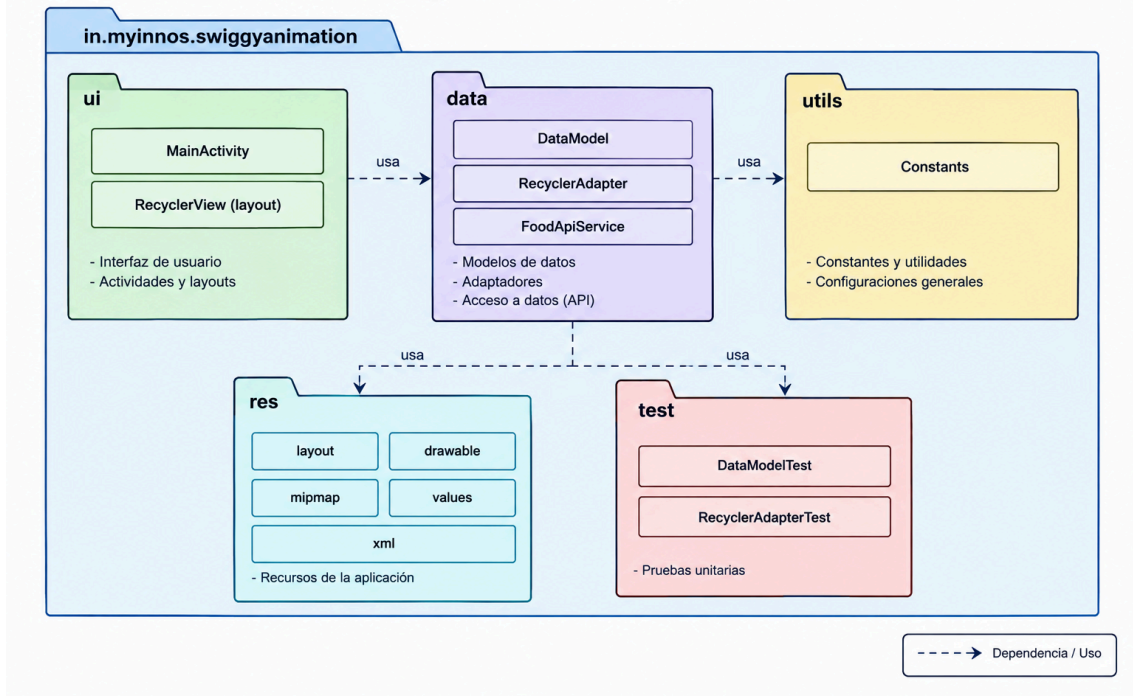
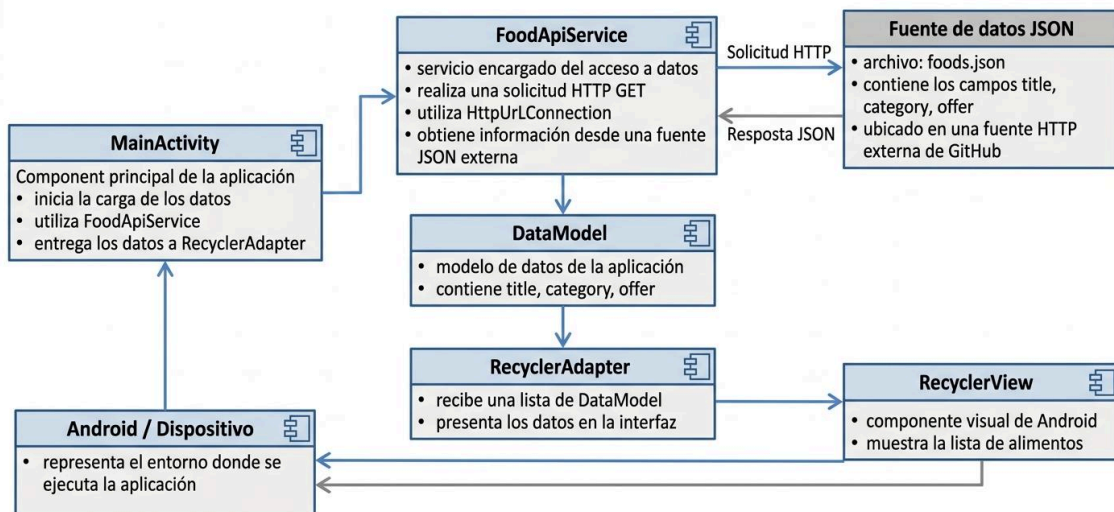


DIAGRAMA DE COMPONENTES

La aplicación está conformada por diferentes componentes que trabajan conjuntamente para presentar la información al usuario.

El componente principal de interacción es `MainActivity`. Los datos son representados mediante `DataModel` y posteriormente administrados por `RecyclerViewAdapter`, que se encarga de relacionar la información con los elementos visuales del `RecyclerView`.

Diagrama de Componentes – AppmovilTres



CAPAS Y UBICACIÓN DE LOS COMPONENTES

Los componentes del proyecto pueden organizarse conceptualmente de la siguiente manera:

Capa de presentación: contiene la actividad principal, layouts XML y elementos visuales utilizados para mostrar la información.

Capa de modelo: contiene DataModel, encargado de representar los datos utilizados por la aplicación.

Capa de adaptación de datos: contiene RecyclerViewAdapter, encargado de relacionar los datos con los elementos visuales del RecyclerView.

Esta organización permite separar responsabilidades y facilita el mantenimiento del código.

METODOLOGÍA DE DESARROLLO

Para el desarrollo se siguió un proceso organizado basado en la identificación de los requerimientos, estructuración del proyecto, implementación de los componentes, organización del código, incorporación de dependencias y elaboración de pruebas unitarias.

El código fuente fue administrado mediante Git, permitiendo registrar los cambios realizados durante el desarrollo.

MAPA DE NAVEGACIÓN

La aplicación cuenta con una actividad principal desde la cual se presenta la información correspondiente a los elementos de la aplicación.

La navegación se encuentra relacionada con la actividad principal y los componentes visuales utilizados para presentar la información.

MÓDULOS Y COMPONENTES DESARROLLADOS

Los principales componentes implementados son:

MainActivity.java

Es la actividad principal de la aplicación y contiene la lógica relacionada con la inicialización y presentación de la interfaz.

DataModel.java

Es el modelo utilizado para almacenar información de cada elemento. Contiene atributos para título, categoría y oferta, junto con sus respectivos métodos de acceso.

RecyclerViewAdapter.java

Es el adaptador utilizado para conectar la información almacenada en los objetos DataModel con los elementos visuales de la lista.

Los archivos XML de la carpeta res contienen los recursos gráficos, colores, estilos y diseños utilizados por la aplicación.

LENGUAJE DE PROGRAMACIÓN

El lenguaje seleccionado para el desarrollo de los módulos es Java.

La implementación utiliza clases, atributos, métodos, constructores y estructuras propias del lenguaje para organizar la lógica de la aplicación.

LIBRERÍAS UTILIZADAS

El proyecto utiliza diferentes librerías para apoyar el desarrollo de la interfaz y la funcionalidad de la aplicación.

Entre las dependencias configuradas se encuentran:

- AndroidX AppCompat.
- Material Components.
- ConstraintLayout.
- ButterKnife.
- JUnit 4.12 para las pruebas unitarias.

Las dependencias se administran mediante Gradle en el archivo app/build.gradle.

FRAMEWORK Y COMPONENTES DE ANDROID

El proyecto utiliza la plataforma Android y sus componentes para desarrollar la aplicación móvil.

Entre los componentes utilizados se encuentra RecyclerView, empleado para presentar información mediante una lista de elementos reutilizables.

La interfaz se define mediante archivos XML ubicados en los recursos de la aplicación.

COMPONENTES REUTILIZABLES

El RecyclerView constituye un componente reutilizable para administrar los elementos que se muestran dentro de la lista.

El uso de DataModel permite representar cada elemento de manera independiente y facilita que el adaptador pueda trabajar con una colección de objetos.

Esta separación permite reutilizar el componente de adaptación para diferentes conjuntos de datos que mantengan la misma estructura.

BUENAS PRÁCTICAS DE PROGRAMACIÓN

Durante la organización del código se aplicaron prácticas orientadas a mejorar su comprensión y mantenimiento.

Entre ellas se encuentran:

- Utilización de nombres representativos para clases y métodos.
- Separación de responsabilidades entre modelo, actividad y adaptador.
 - Organización del código mediante paquetes.
- Uso de comentarios para explicar partes relevantes del código.
- Administración de dependencias mediante Gradle.

- Implementación de pruebas unitarias para componentes que pueden ser evaluados independientemente.
- Uso de Git para el control de versiones.

PATRONES Y ORGANIZACIÓN DE COMPONENTES

La aplicación utiliza una separación entre los datos y su representación visual mediante DataModel y RecyclerView.

Esta organización permite separar las responsabilidades de los componentes y facilita el mantenimiento del código.

El patrón utilizado para la presentación de colecciones de elementos corresponde al uso de Adapter mediante RecyclerView, que permite conectar los datos con los elementos visuales del RecyclerView.

CONTROL DE VERSIONES GIT

El código fuente del proyecto se encuentra almacenado en un repositorio GitHub.

Repositorio:

<https://github.com/stronck/AppmovilTres>

El repositorio contiene los archivos fuente, configuración del proyecto Android, recursos, Gradle Wrapper y el archivo APK correspondiente al proyecto.

El uso de Git permite conservar el historial de modificaciones y mantener organizado el desarrollo del proyecto.

PRUEBAS UNITARIAS

Como parte de la evidencia se implementaron pruebas unitarias utilizando JUnit 4.12.

Las pruebas se encuentran ubicadas en:

app/src/test/java/in/myinnos/swiggyanimation/

Se desarrollaron pruebas para los componentes DataModel y RecyclerView.

Las pruebas fueron implementadas como pruebas unitarias locales, evitando depender de la interfaz gráfica o de un dispositivo físico para las validaciones realizadas.

PRUEBAS UNITARIAS DE DATAMODEL

Se creó la clase:

DataModelTest.java

Esta clase contiene pruebas para verificar que los valores suministrados al modelo sean recuperados correctamente mediante sus métodos correspondientes.

Se verifican los valores de:

- Título.

- Categoría.
- Oferta.

También se incluye una prueba utilizando valores vacíos para comprobar que el modelo conserve correctamente dichos valores.

PRUEBAS UNITARIAS DE RECYCLERADAPTER

Se creó la clase:

`RecyclerViewAdapterTest.java`

Esta clase verifica la cantidad de elementos administrados por el adaptador.

Se incluyen dos casos:

- Una lista con tres elementos, donde se espera que el adaptador informe una cantidad de tres elementos.
- Una lista vacía, donde se espera que el adaptador informe una cantidad de cero elementos.

Estas pruebas permiten validar el comportamiento básico del método `getItemCount()`.

ESTRUCTURA DE LAS PRUEBAS

La estructura de pruebas implementada es:

`app/src/test/java/in/myinnos/swiggyanimation/DataModelTest.java`

`app/src/test/java/in/myinnos/swiggyanimation/RecyclerViewAdapterTest.java`

En total se implementaron cuatro casos de prueba:

- Dos pruebas para `DataModel`.
- Dos pruebas para `RecyclerViewAdapter`.

Las pruebas se encuentran incorporadas dentro del código fuente del repositorio.

CONFIGURACIÓN DE LAS PRUEBAS

Para utilizar JUnit se agregó la dependencia correspondiente en el archivo `app/build.gradle`:

```
testImplementation 'junit:junit:4.12'
```

Esta configuración permite que el proyecto disponga de pruebas unitarias locales utilizando JUnit.

EVIDENCIA DEL CÓDIGO FUENTE

El repositorio contiene los archivos Java principales, archivos XML de interfaz y recursos, además de los archivos necesarios para la configuración del proyecto Android.

Entre los archivos principales se encuentran:

MainActivity.java

DataModel.java

RecyclerViewAdapter.java

DataModelTest.java

RecyclerViewAdapterTest.java

CONFIGURACIÓN DE GRADLE

El proyecto utiliza Gradle para administrar la configuración de compilación y las dependencias.

El módulo principal contiene el archivo:

app/build.gradle

En este archivo se encuentran configurados el SDK de Android, las versiones de la aplicación, las opciones de compilación y las librerías utilizadas.

ARCHIVO APK

El repositorio contiene un archivo APK correspondiente a la aplicación:

apk/app-debug.apk

Este archivo corresponde al producto compilado incluido como parte de los entregables del proyecto.

DOCUMENTACIÓN DEL AMBIENTE DE DESARROLLO Y PRUEBAS

El ambiente del proyecto está basado en Android, Java, Gradle y las herramientas de desarrollo correspondientes a Android.

Para las pruebas unitarias se utiliza JUnit 4.12 mediante la configuración testImplementation del módulo de aplicación.

La estructura del proyecto permite separar el código principal de las pruebas mediante las carpetas:

app/src/main/

y

app/src/test/

RESULTADOS DEL DESARROLLO

Como resultado de la implementación se dispone de un proyecto Android organizado que contiene:

- Código fuente Java.
- Actividad principal.
- Modelo de datos.
- Adaptador RecyclerView.
- Recursos XML.
- Configuración Gradle.
- Dependencias necesarias para la aplicación.
- Pruebas unitarias implementadas.
- Repositorio GitHub.
- Archivo APK.

Los componentes se encuentran organizados dentro de una estructura de proyecto Android y se encuentran disponibles en el repositorio definido para la evidencia.

Conclusiones

El desarrollo de la evidencia permitió implementar y organizar los componentes principales de una aplicación móvil Android utilizando Java.

Se estructuró el proyecto mediante paquetes y componentes diferenciados, se configuraron las dependencias mediante Gradle y se utilizó RecyclerView junto con un adaptador para presentar la información de la aplicación.

También se implementaron pruebas unitarias mediante JUnit para validar el funcionamiento básico del modelo de datos y del adaptador.

Finalmente, el código fuente y los demás archivos relacionados con el proyecto se organizaron en un repositorio GitHub, incluyendo el archivo APK correspondiente como parte de los productos de la evidencia.